SELECTED WORKS / AI·서비스

Video Translator

영상의 외국어 음성을 편집 가능한 자막으로

영상 업로드·비동기 음성 인식·번역·편집을 거쳐 시간 정보가 유지된 SRT/VTT 자막을 만드는 MVP입니다.

https://github.com/MaoEmong/video-translator

웹 상세 소개 보기

Python 3.12 · FastAPI · PostgreSQL · Redis · MinIO · FFmpeg · OpenAI

입력·출력	영상 파일 → 편집 가능한 SRT·VTT
실행 구조	FastAPI API + Redis Queue + 별도 Python Worker
저장	PostgreSQL 작업 상태 · MinIO/S3 객체 파일
AI 실행 방식	whisper-1 timed STT · Responses API 구조화 번역

01 / CONTEXT

어떤 문제에서 시작했나요?

영상 번역은 문장을 다른 언어로 바꾸는 것만으로 끝나지 않습니다. 음성을 추출하고 인식하는 동안 시간이 흐르며, 번역 후에도 화면에 표시할 구간이 원본 음성과 맞아야 합니다. 파일 전체를 API 서버가 중계하거나 긴 처리를 HTTP 요청 안에 묶으면 요청 수명과 미디어 작업 수명이 충돌합니다.

Video Translator는 업로드·작업 등록·단계 처리·편집·내보내기로 이 과정을 나눴습니다. 외부 AI가 반환한 JSON은 요청한 세그먼트와 일치하는지 검사하고, 원문 시간 정보는 자막 생성까지 유지합니다. 웹 API·저장소·작업 큐·AI 응답을 연결하되 오래 걸리는 처리와 사용자의 편집을 각자의 수명에 맞게 다루도록 구성했습니다.

02 / PRODUCT FLOW

기능과 사용자 흐름

비동기 영상 처리

Object Storage로 영상을 올리고 Redis 작업 큐로 음성 추출과 후속 처리를 수행합니다.

음성 인식과 번역

FFmpeg로 오디오를 정리하고 Whisper 음성 인식과 구조화된 번역 응답을 사용합니다.

편집과 내려받기

번역문·표시 시간 편집, 번역 버전 관리와 SRT·VTT 내보내기를 제공합니다.

한 번의 사용을 따라가 보면

01 업로드 세션 만들기

영상 업로드를 요청해 storage key와 signed URL을 받습니다. 브라우저가 객체 저장소에 직접 전송하고 업로드 완료 검증을 요청합니다.

02 번역 작업 등록하기

준비된 영상과 목표 언어·번역 스타일로 Job을 만듭니다. API 응답과 장시간 처리를 분리하고 작업 ID로 상태를 조회합니다.

03 단계별 진행 확인하기

Worker가 검증→오디오 추출→STT→정규화→번역→자막 생성으로 메시지를 이어 보냅니다. UI에서 현재 단계·진행률을 보고 실패 시 해당 상태를 확인합니다.

04 문장과 표시 시간 편집하기

AI 번역 결과를 세그먼트 단위로 검토하고 번역문·표시 시간을 수정합니다. 기본 시간은 원문 STT에서 가져오고 편집 시간이 있으면 내보내기에서 우선 사용합니다.

05 자막 파일 내려받기

선택한 버전에서 SRT 또는 VTT Artifact를 만들고 만료 URL로 내려받습니다. 자막 파일 생성과 번역 영상 렌더링·음성 더빙은 별도 범위입니다.

03 / ARCHITECTURE

구조와 책임을 나눈 방식

01 API가 상태를 만들고 Worker가 처리

FastAPI는 업로드·작업 생성·상태 조회·세그먼트 편집·내보내기 요청을 담당합니다. 작업과 단계 상태는 PostgreSQL에 기록하고 Redis로 다음 처리 메시지를 전달합니다. API와 Worker는 같은 코드베이스의 별도 프로세스이며 UI는 polling으로 진행 상태를 확인합니다.

02 서버를 거치지 않는 원본 전송

API가 만료 시각을 가진 업로드 signed URL을 발급하면 브라우저가 private MinIO/S3에 직접 파일을 보냅니다. 완료 검증 후 번역 작업을 만들고, 결과 자막도 만료되는 signed download URL로 전달합니다. API는 바이트 중계 대신 파일 메타데이터와 작업 상태를 관리합니다.

03 시간 정보를 가진 AI 파이프라인

Worker는 FFmpeg로 mono 16kHz WAV를 만든 뒤 whisper-1의 segment 시간 정보를 밀리초로 정규화합니다. 번역 Adapter에는 세그먼트 ID·원문을 보내고 구조화 응답의 ID 집합·중복·빈 문장을 검사합니다. 원본 시간과 번역 텍스트를 연결해 자막을 생성합니다.

04 번역 버전과 파일 산출물의 분리

TranscriptSegment는 원문과 기본 시간, TranslationSegment는 번역문과 선택적인 표시 시간을 가집니다. TranslationVersion과 SubtitleArtifact를 나누어 편집 결과에서 SRT/VTT를 다시 만들 수 있도록 구성했습니다. 최종 자막은 UTF-8 바이트로 저장하고 크기·체크섬을 기록합니다.

04 / ENGINEERING CASES

구현하면서 선택한 것

CASE 01 / 긴 작업의 상태를 PostgreSQL에 남기기

마주한 문제

FFmpeg·STT·번역을 동기 요청으로 묶으면 timeout과 응답 지연이 생기고 처리 중 어디에 있는지 알기 어렵습니다.

선택과 구현

API는 작업을 기록하고 Redis에 메시지를 넣습니다. Worker는 단계 상태를 queued/running/completed/failed로 남기며, 한 단계를 마치면 다음 메시지를 등록합니다. 영속적인 상태의 기준은 PostgreSQL로 두었습니다.

남은 비용과 한계

별도 프로세스와 Redis 운영이 필요합니다. MVP에서는 Celery/RabbitMQ의 복잡성을 줄인 대신 자동 재시도·Dead Letter Queue·멱등성 정책은 후속 과제로 남겼습니다.

ADR-002 비동기 처리 선택

CASE 02 / 번역 모델에 자막 시간을 맡기지 않기

마주한 문제

번역 응답이 문장을 합치거나 ID를 누락하면 원문 구간과 번역문을 연결할 수 없습니다. 텍스트만 있는 STT 결과로는 자막의 정확한 표시 시간을 만들기 어렵습니다.

선택과 구현

STT에서 timed segment를 확보하고 번역은 ID와 문장 변환에 집중시켰습니다. strict JSON schema를 요청한 뒤에도 ID 집합·개수·빈 번역문을 다시 검사하고, 자막 생성은 원문 시간 또는 사용자의 편집 시간을 사용합니다.

남은 비용과 한계

timed segment를 지원하는 모델 기능에 의존하며 장시간 chunking·화자 분리는 제외했습니다. 형식 검사 통과는 번역 의미의 정확성을 보장하지 않아 사람의 편집 단계를 유지합니다.

OpenAI Adapter의 시간·응답 검사

CASE 03 / 대용량 전송을 객체 저장소로 직접 보내기

마주한 문제

API 서버가 영상 전체를 받아 다시 저장하면 메모리·연결 시간·전송 부담이 커지고, 공개 고정 URL은 파일 접근 수명을 통제하기 어렵습니다.

선택과 구현

private 객체 저장소에 대해 PUT/GET presigned URL을 발급합니다. 업로드에는 Content-Type과 TTL을 지정하고 head_object로 파일 메타데이터를 조회합니다. 원본·정규화 오디오·자막은 key를 통해 연결합니다.

남은 비용과 한계

브라우저 전송·저장소 설정과 업로드 완료 검증을 별도로 다뤄야 합니다. 만료 URL이 사용자 인증·소유권을 대신하지 않으며 운영 보관 기간·자동 삭제·다중 사용자 권한은 추가 설계가 필요합니다.

S3 gateway와 signed URL

05 / ROLE & CONTRIBUTION

맡은 역할과 기여

영상 업로드부터 번역 작업 등록, Worker의 단계 처리, 자막 편집과 다운로드까지 이어지는 비동기 흐름을 구성했습니다. API는 요청과 상태 조회를, Worker는 오래 걸리는 미디어 처리를 맡도록 나누고 PostgreSQL에 작업 상태를 남겼습니다.

파일 전송에는 객체 저장소의 만료 URL을 사용하고, AI 응답은 세그먼트 ID·개수·빈 번역문을 검사한 뒤 반영하도록 연결했습니다. 원문 시간과 사용자의 편집 시간을 자막 생성에 적용하고, 큐·저장소·외부 AI를 선택한 이유와 남은 과제는 ADR로 정리했습니다.

06 / EVIDENCE & LIMITS

무엇을 확인했고, 무엇이 남았나요?

프로젝트 검증 당시의 실행 기록과 코드에 구성한 테스트 항목을 정리했습니다. 각 항목에 실행 조건과 추가 확인이 필요한 범위를 함께 적었습니다.

Mock 기반 자동 검사

프로젝트 README에 정리한 자동 테스트 결과는 19개 통과입니다. 업로드·작업 생성 API와 번역→자막 파이프라인을 다뤘습니다.

외부 서비스를 Mock으로 대체한 검사입니다. 실제 번역 품질과 장시간 영상 처리는 별도 평가가 필요합니다.

검증 근거 보기

외부 구성요소 연결

Docker·MinIO·FFmpeg·OpenAI를 연결한 smoke test 완료 기록을 README에 남겼습니다.

간단한 연결·실행 확인 범위입니다. 원시 실행 로그는 별도로 공개하지 않았으며, 운영 안정성과 번역 품질의 정량 평가는 남아 있습니다.

검증 근거 보기

단계 처리와 자막 생성 코드

파이프라인에 단계별 메시지 전달, 시간 정보가 있는 세그먼트 저장, 편집 시간 우선 적용, 자막 바이트와 체크섬 생성을 구현했습니다.

연결된 소스에서 처리 흐름을 확인할 수 있습니다. 장애 복구·중복 메시지·동시 편집·장시간 입력의 실행 검증은 후속 과제입니다.

검증 근거 보기

현재 범위와 남은 과제

• 짧은 영상과 로컬 단일 사용자 실행을 기준으로 만든 MVP입니다. 인증·다중 사용자 권한과 운영 배포 검증은 남아 있습니다.

• 자동 재시도·Idempotency-Key·Dead Letter Queue는 미구현입니다. 중복 메시지와 실패 작업의 안전한 재처리 정책을 추가해야 합니다.

• 장시간 영상의 chunk 분할·화자 분리·번역 영상 렌더링·음성 더빙은 현재 범위에서 제외했습니다.

• 운영 모니터링·quota·결제·자동 보관 삭제와 번역 품질의 정량 평가는 후속 과제입니다.

다음에 보완할 것

• 장시간 입력의 크기·길이·비용 상한을 먼저 정하고 chunking·재시도·멱등성·실패 메시지 처리를 설계합니다.

• 다중 사용자 인증과 소유권·파일 lifecycle을 정하고 편집 충돌·언어별 번역 품질의 평가 기준을 추가합니다.

07 / SOURCE MATERIAL

코드와 개발 기록

아래 공개 자료에서 구현 범위와 설계 판단을 확인할 수 있습니다. 링크를 누르면 원문으로 이동합니다.

01 기능·실행·기존 검증 기록

https://github.com/MaoEmong/video-translator/blob/main/README.md

02 Worker의 단계 처리 구현

https://github.com/MaoEmong/video-translator/blob/main/worker/pipeline.py

03 OpenAI STT·번역 Adapter

https://github.com/MaoEmong/video-translator/blob/main/backend/app/providers/openai_adapters.py

04 signed URL 저장소 구현

https://github.com/MaoEmong/video-translator/blob/main/backend/app/storage/s3.py

05 Queue·Worker 결정과 대안

https://github.com/MaoEmong/video-translator/blob/main/documents/adr/ADR-002-redis-queue-and-worker.md

06 운영 확장 검토 사항

https://github.com/MaoEmong/video-translator/blob/main/documents/13-open-decisions.md